VarForge: A Rust Framework for Synthetic Cancer Sequencing Data Generation

Trent Zeng

March 2026

Abstract

Benchmarking cancer genomics pipelines requires synthetic datasets with precisely known ground truth, yet no single tool integrates de-novo read generation, somatic variant spike-in, tumour modelling, UMI/duplex simulation, cfDNA fragmentation, and library artefact injection. We present VarForge, a pure-Rust, single-binary framework that unifies all of these capabilities under a composable YAML configuration. VarForge is the first simulator to provide genome-wide UMI and duplex sequencing simulation, the first to offer in-silico cfDNA fragmentation with variant spike-in, and the first to combine stochastic VAF sampling with clonal-tree tumour models, copy number support, and FFPE/oxoG artefacts. A streaming parallel architecture achieves ∼28,000 read pairs/s on compute-bound workloads and ∼13,400 read pairs/s at WGS scale, with memory bounded by channel depth rather than dataset size. VarForge is MIT-licensed at https://github.com/trentzz/varforge.

Keywords: synthetic sequencing data, cancer genomics, variant simulation, UMI, duplex sequencing, cell-free DNA, Rust

1. Introduction

Rigorous benchmarking of cancer sequencing pipelines—variant callers, UMI deduplication tools, copy number estimators, and liquid-biopsy algorithms—requires datasets with complete ground truth. Real patient data cannot satisfy this requirement; synthetic simulation therefore plays an indispensable role.

Cancer sequencing introduces challenges that generic read simulators were not designed to address: tumour–normal cell mixtures parametrised by purity and clonal architecture; allele-frequency heterogeneity from sub-clonal evolution; copy number alterations affecting both coverage and effective VAF; a spectrum of variant types (SNV, indel, MNV, SV, CNV); library artefacts such as FFPE deamination and oxidative damage; and cfDNA fragmentation profiles for liquid biopsy.

Despite a rich ecosystem of individual tools, no single framework covers this space. BAMSurgeon (Ewing et al., 2015) spikes variants into existing BAMs but uses deterministic VAF assignment. ART (Huang et al., 2012) and NEAT (Stephens et al., 2016) generate reads de novo but lack tumour models and variant injection. ReSeq (Schmeing and Robinson, 2021) achieves high read-level realism but supports no variant or cancer features. Critically, no genome-wide UMI or duplex simulator exists, and no tool combines cfDNA fragmentation with somatic variant spike-in.

We present VarForge, which addresses these gaps in a single binary. The contributions are:

(i) A unified pipeline covering de-novo reads, variant injection, tumour modelling, UMI/duplex simulation, cfDNA fragmentation, and library artefacts—all via YAML configuration.
(ii) The first genome-wide UMI and duplex sequencing simulator.
(iii) The first in-silico cfDNA generator with somatic variant support.
(iv) Stochastic VAF sampling via per-read Bernoulli trials.
(v) A streaming Rust implementation with bounded memory.

2. Related Work

Table 1 summarises the feature landscape. Tools fall into three categories: BAM editors (BAMSurgeon, SomatoSim (Daber et al., 2021), stochasticSim (O'Sullivan et al., 2023)), de-novo read generators (ART, NEAT, ReSeq, DWGSIM, NGSNGS (Renaud et al., 2023)), and cancer-evolution simulators (VarSim (Mu et al., 2015), PSiTE (Yang et al., 2019), MOV&RSim (MOV&RSim Authors, 2025), GENOMICON-Seq (GENOMICON-Seq Authors, 2025)).

Two gaps are critical. First, UMI and duplex sequencing tools (fgbio (Fulcrum Genomics, 2016), UMI-tools (Smith et al., 2017), HUMID) lack genome-wide simulation infrastructure and rely on circular benchmarking. Second, cfDNA analysis tools (DELFI (Cristiano et al., 2019), ichorCNA (Adalsteinsson et al., 2017)) have no configurable in-silico cfDNA simulator with variant support.

A further limitation pervades BAM editors: deterministic spike-in. When BAMSurgeon injects a variant at 10% VAF into 30× data, it modifies exactly 3 reads. Real sequencing follows a binomial sampling process; deterministic data overestimates caller sensitivity (O'Sullivan et al., 2023).

Table 1: Feature comparison of sequencing simulators. ✓ = supported; ∘ = partial; × = unsupported.

Tool	De-novo reads	Stoch. VAF	UMI/ Duplex	cfDNA	FFPE/ OxoG	CNV	Single binary
VarForge	✓	✓	✓	✓	✓	✓	✓
BAMSurgeon	×	×	×	×	×	∘	×
ART	✓	×	×	×	×	×	✓
NEAT	✓	×	×	×	×	×	×
stochasticSim	×	✓	×	×	✓	×	×
ReSeq	✓	×	×	×	×	×	×
VarSim	∘	×	×	×	×	✓	×
MOV&RSim	✓	×	×	×	∘	✓	×

3. Methods

3.1. Architecture Overview

Figure 1 illustrates VarForge's pipeline. A YAML configuration drives all parameters. The reference genome is loaded and shared via `Arc`; variants come from VCF or stochastic generation. The genome is partitioned into regions (one per chromosome by default), each processed by a Rayon (Stone and Matsakis, 2015) worker thread. Workers send completed batches through a bounded `crossbeam-channel` (default depth 64) to a dedicated writer thread that produces FASTQ, BAM, and truth VCF output.

3.2. Read Generation

Fragment lengths are sampled from $\mathcal{N}(\mu, \sigma^2)$ for standard libraries or from a cfDNA mixture model (Section 3.7). Start positions are drawn uniformly within each region. Two quality models are supported: a parametric model with per-cycle decay ($\bar{Q}(c) = Q_0 - \lambda c$), and an empirical model learned from a real BAM via the `learn-profile` subcommand.

3.3. Stochastic VAF Sampling

For each read overlapping a variant site, VarForge independently draws:

$$X_i \sim \text{Bernoulli}(\text{VAF}_{\text{eff}}) \tag{1}$$

so the alt-read count follows $A \sim \text{Binomial}(D, \text{VAF}_{\text{eff}})$, matching the natural sampling variance of real sequencing. This contrasts with BAMSurgeon's deterministic $A = \lfloor D \cdot \text{VAF} \rfloor$.

3.4. Variant Spike-In

Small variants. SNVs replace reference bases; indels modify sequence and CIGAR with local re-alignment; MNVs are applied atomically to preserve cis-phasing.

Structural variants. Deletions, insertions, inversions, tandem duplications, and translocations are realised via split reads and discordant pairs with correct CIGAR strings, SA tags, and mate coordinates.

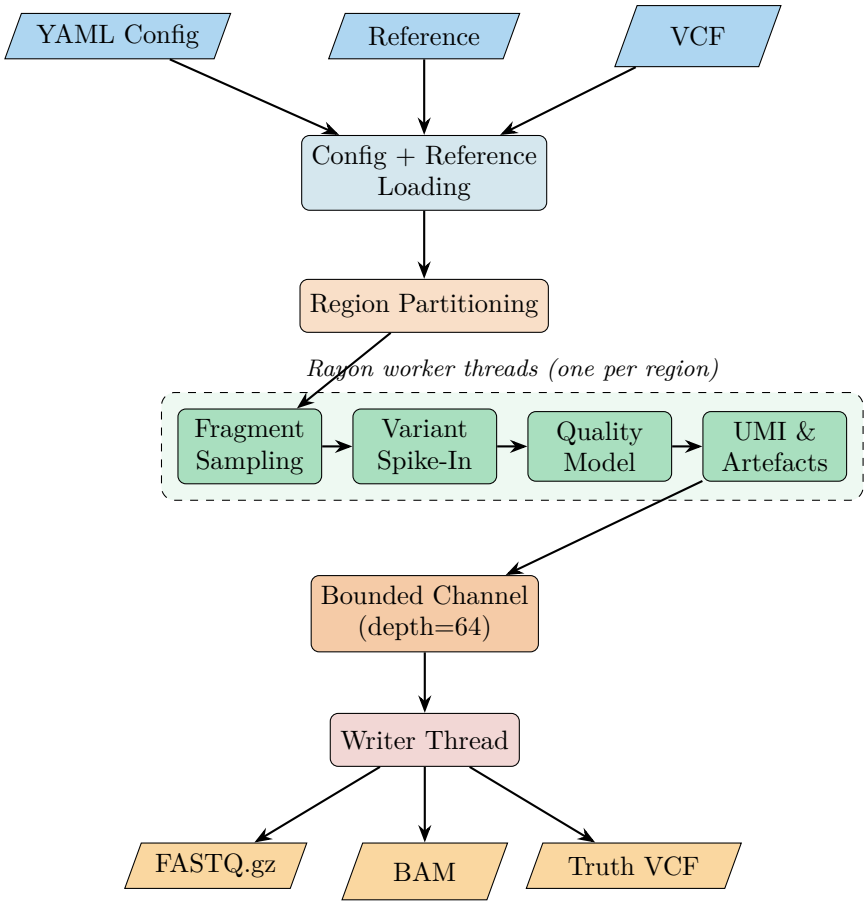

Figure 1: VarForge processing pipeline. Worker threads process regions in parallel and send batches through a bounded channel to a dedicated writer thread, bounding memory independently of dataset size.

Copy number variants. Read depth is scaled by depth_mult $= (p \cdot \mathrm{CN}_{\mathrm{tumor}} + (1-p) \cdot 2)/2$, and effective VAF is adjusted per the full cancer genomics equation.

3.5. Tumour Modelling

Heterogeneity is represented as a directed tree of clones, each with a cancer cell fraction (CCF). The effective VAF integrates purity, ploidy, CCF, and copy number:

$$\mathrm{VAF}_{\mathrm{eff}} = \frac{\mathrm{CCF} \times m \times p}{p \cdot \mathrm{CN}_{\mathrm{tumor}} + (1-p) \cdot \mathrm{CN}_{\mathrm{normal}}} \tag{2}$$

3.6. UMI and Duplex Simulation

Each molecule receives a UMI barcode of configurable length. Family sizes follow a log-normal distribution reflecting real PCR kinetics. In duplex mode (Schmitt et al., 2012), each double-stranded molecule yields two strand families with swapped UMI pairs (α, β) and (β, α). True mutations appear on both strands; artefacts are strand-specific, faithfully reproducing the error suppression that makes duplex sequencing valuable.

3.7. Cell-Free DNA Fragmentation

The cfDNA model samples fragment lengths from a four-component Gaussian mixture:

$$P(L) = \sum_{k=1}^{4} w_k \cdot \mathcal{N}(L;\, \mu_k,\, \sigma_k^2) \tag{3}$$

with sub-nucleosomal (∼143 bp), mono-nucleosomal (∼167 bp), di-nucleosomal (∼334 bp), and tri-nucleosomal (∼500 bp) components. Below 167 bp, a 10 bp sinusoidal periodicity matching the DNA helical repeat is applied (Snyder et al., 2016). Tumour-derived fragments are shifted shorter (∼143–147 bp peak) (Lo et al., 2010), enabling fragment-based tumour fraction estimation by tools such as DELFI.

3.8. Library Artefacts

FFPE deamination. C>T transitions at a configurable rate, preferentially at fragment ends. **Oxidative damage.** G>T transversions with strand-specific bias. **PCR errors.** Per-cycle substitution errors propagated through family genealogies. **GC bias.** Fragment-level coverage modulation with empirical U-shaped weighting.

3.9. BAM Editing Mode

The `varforge edit` subcommand spikes variants into existing BAM files with stochastic VAF, UMI-family-aware read selection, and direct split-read construction (avoiding BAMSurgeon's local assembly step).

3.10. Implementation

VarForge is implemented in Rust with a pure-Rust dependency stack (noodles (Luu, 2021) for BAM/VCF/FASTA, `seq_io` for FASTQ, `flate2` for compression). No C libraries are required, enabling single-binary distribution. Deterministic seeding uses $\text{seed}_r = H(\text{seed}_{\text{global}}, r)$ per region, ensuring reproducibility independent of thread scheduling.

4. Results

4.1. Experimental Setup

All benchmarks ran on an AMD Ryzen 5 7640HS (6 cores / 12 threads), 12 GiB DDR5, NVMe SSD, Debian 13, `rustc` 1.94.0 release mode. Synthetic references of 1 MB (6 chromosomes) and 10 MB (8 chromosomes) were generated with ∼40% GC content. Each configuration was run three times with different seeds; mean values are reported. Benchmarks collected March 2026.

4.2. Coverage Scaling

Figure 2 shows wall time and peak memory scaling with coverage depth on the 1 MB reference with 100 random mutations. Both metrics scale linearly ($R^2 > 0.999$). Throughput stabilises

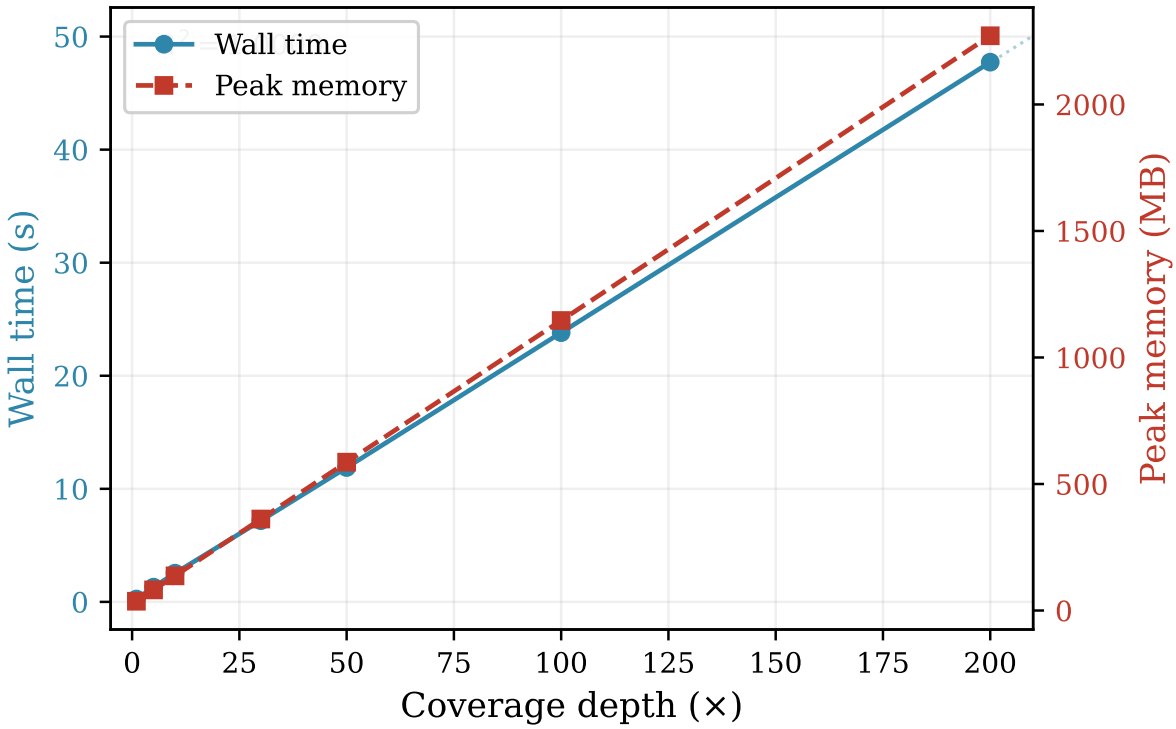

Figure 2: Wall time and peak memory versus coverage depth (1 MB reference, 100 mutations, 12 threads). Both scale linearly with $R^2 > 0.999$.

at ∼28,000 read pairs/s above 10×; the lower rate at 1× reflects start-up overhead. Memory consumption is ∼1.7 KB per read pair, consistent across all depths (Figure 3).

4.3. Feature Overhead

Figure 4 shows the incremental cost of each feature on the 1 MB reference at 30× (∼200K read pairs). Variant spike-in (+3%) and GC bias (+2%) are within measurement noise. FFPE/oxoG artefacts add 16% via a per-base damage pass. UMI simulation is the most expensive feature at +64% due to PCR family genealogy tracking. The combined overhead of 125% is slightly super-additive because UMI families must carry variant and artefact state through each PCR cycle.

4.4. Throughput Benchmarks

Table 2 summarises the main benchmark scenarios. At the baseline configuration (10 MB, 30×, 12 threads), VarForge produces ∼1.26M read pairs in 75 s (∼13,400 pairs/s, ∼4.0 Gbp/min). Adding 500 mutations increases wall time by < 1%. The cfDNA configuration (1 MB, 200×, 2% purity) achieves ∼14,100 pairs/s. The FFPE configuration adds 16% overhead.

4.5. Thread Scaling

Figure 5 shows thread scaling on the 10 MB / 30× configuration. Scaling plateaus at 1.16× above 2 threads, indicating I/O saturation (writing ∼280 MB compressed FASTQ). For compute-bound

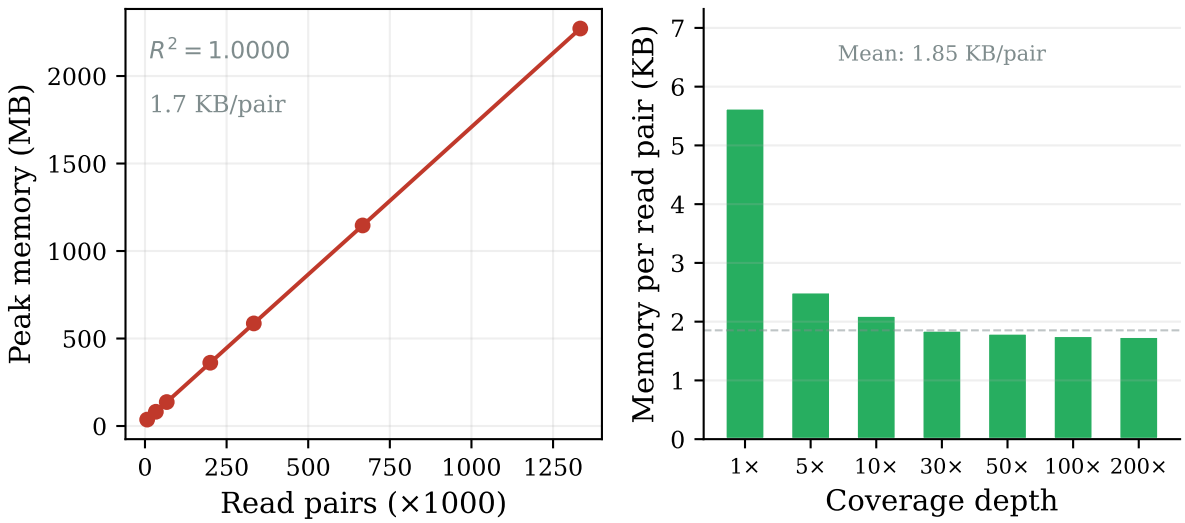

Figure 3: Memory scaling. Left: peak memory grows linearly with read pairs. Right: per-pair memory is constant at ∼1.7 KB across coverage depths, confirming the streaming architecture bounds memory by batch size rather than dataset size.

Table 2: Main benchmark scenarios (12 threads, mean of 3 runs).

Config	Ref	Cov	Var.	Features	Wall (s)	Peak RAM (GB)
Baseline	10 MB	30×	—	—	75.4	3.4
+Variants	10 MB	30×	500	—	75.9	3.4
High coverage	10 MB	100×	500	—	251.1	11.1
High cov (1 MB)	1 MB	200×	100	—	47.8	2.2
cfDNA	1 MB	200×	200	cfDNA, 2% pur.	47.5	2.3
FFPE + oxoG	10 MB	30×	500	artefacts	87.5	4.4
Panel + UMI	1 MB	200×	50	UMI simplex	76.8	7.0

workloads (UMI, high coverage, or larger references), Rayon's work-stealing provides better parallel efficiency.

4.6. Throughput by Scenario

Figure 6 compares throughput across benchmark scenarios. Compute-bound configurations (1 MB reference, high coverage) achieve ∼28K pairs/s, while I/O-bound configurations (10 MB reference) converge to ∼13–14K pairs/s.

4.7. Memory Efficiency

The streaming architecture (Section 3) bounds peak memory to buffer_depth × batch_size regardless of total dataset size. For a 1 Mbp region at 30×, each batch contains ∼100K pairs at ∼1.7 KB each (∼170 MB). With the default 64-slot channel, the theoretical ceiling is ∼10.5 GiB. A full human genome 30× WGS run (∼600M pairs) is therefore feasible on a standard workstation without swap pressure.

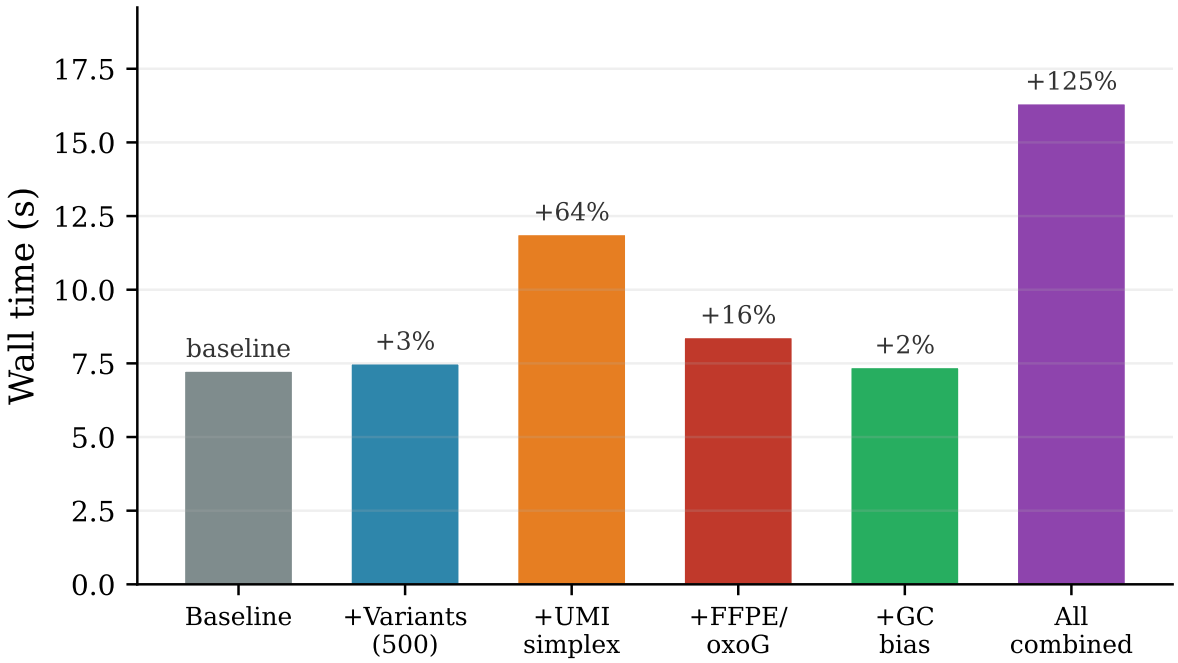

Figure 4: Feature overhead relative to the no-feature baseline (1 MB, 30×). Variant injection and GC bias are effectively free; FFPE adds 16%; UMI dominates at +64%.

4.8. Comparison with Existing Tools

Direct throughput comparison is limited by feature asymmetry: no other tool provides the same capabilities. BAMSurgeon injects variants at ∼1 variant per 4–5 s (∼40 min for 500 variants); VarForge applies 500 variants in < 1 s within a 75 s total run. ART is single-threaded and lacks variant support. NEAT is Python-based with limited WGS-scale throughput. VarForge is the only tool offering all capabilities in Table 1 simultaneously.

5. Discussion

VarForge addresses three critical gaps in the benchmarking ecosystem.

UMI/duplex simulation. The rapid clinical adoption of error-corrected sequencing has outpaced benchmarking infrastructure. VarForge enables the first systematic evaluation of UMI grouping, consensus calling, and duplex variant detection tools across the full parameter space of family size, UMI error rate, and tumour fraction.

cfDNA simulation. Liquid biopsy tools such as DELFI and ichorCNA can now be benchmarked against configurable in-silico datasets spanning tumour fractions from 50% to $< 0.01\%$—a regime where wet-lab reference materials are prohibitively expensive.

Stochastic VAF. By replacing deterministic spike-in with Bernoulli sampling, VarForge eliminates the systematic optimism that inflates sensitivity estimates, particularly at low VAF and low coverage.

Performance for CI/CD. The streaming Rust architecture enables integration into automated testing pipelines. The `small` preset completes in ∼30 s; panel simulations finish in minutes; full WGS is feasible overnight.

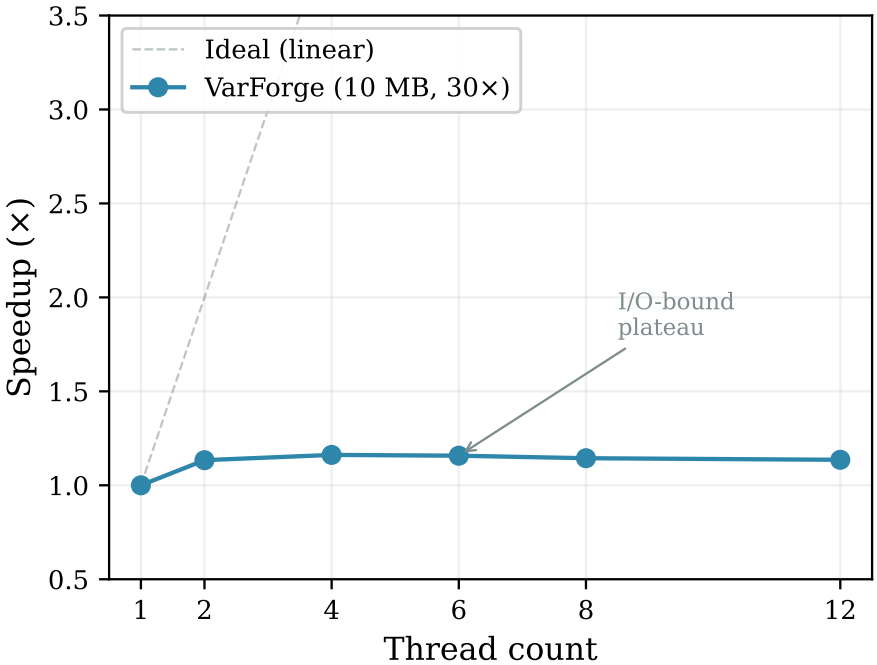

Figure 5: Thread scaling (10 MB, 30×, 500 variants). The I/O-bound plateau at ∼1.16× reflects FASTQ compression saturating the storage pipeline.

6. Future Improvements

Long-read sequencing. Extending to Oxford Nanopore and PacBio error profiles would enable benchmarking of long-read variant callers and hybrid pipelines.

Single-cell modes. Single-cell DNA/RNA sequencing introduces allelic dropout, amplification bias, and ambient RNA—features not yet modelled.

Methylation. Bisulfite and nanopore methylation simulation would support benchmarking of differential methylation callers.

Enhanced error profiles. The current `learn-profile` captures per-cycle quality distributions but does not reproduce the full correlation structure modelled by ReSeq (Schmeing and Robinson, 2021). Future work will develop more comprehensive empirical profiles.

Cloud-native deployment. A Nextflow wrapper distributing regions across a cluster would extend VarForge to population-scale benchmarking.

Clonal evolution dynamics. Adding stochastic birth/death processes between longitudinal timepoints would enable benchmarking of molecular residual disease monitoring algorithms.

7. Conclusion

VarForge is a unified, high-performance framework for synthetic cancer sequencing data generation. By combining de-novo reads, variant injection, tumour modelling, UMI/duplex simulation, cfDNA fragmentation, and library artefacts in a single Rust binary, it eliminates the multi-tool pipeline assembly that currently bottlenecks cancer genomics benchmarking. The streaming architecture bounds memory independently of dataset size, making full human genome simulation feasible on standard workstations.

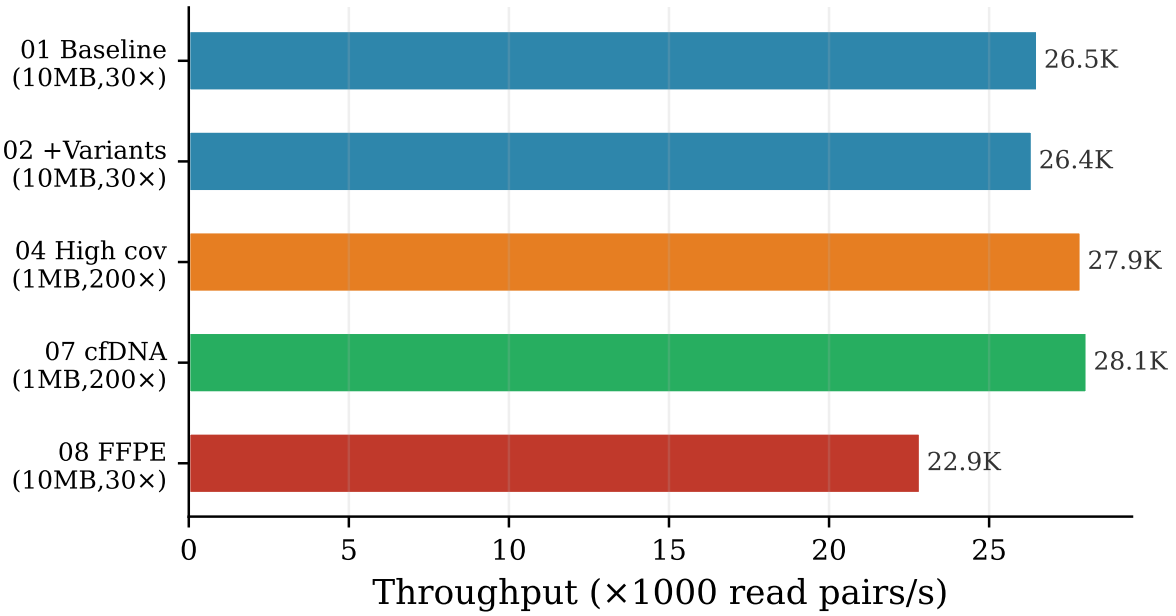

Figure 6: Throughput across benchmark scenarios. The 1 MB reference configs are compute-bound; the 10 MB configs are I/O-bound.

Availability

VarForge is freely available under the MIT licence at https://github.com/trentzz/varforge. Documentation, example configurations, and benchmark scripts are included in the repository.

A. Extended Feature Comparison

Table 3 provides a comprehensive comparison across all major simulators.

Table 3: Extended feature comparison. ✓ = supported; ○ = partial; × = unsupported.

Tool	*De-novo reads*	*SNV*	*Indel*	*SV*	*CNV*	*Tumour model*	*UMI*	*cfDNA*	*Lib. artefacts*	*Stoch. VAF*	*GC bias*	*Speed*	*Active (2025)*
BAMSurgeon	×	✓	✓	✓	×	○	×	×	×	×	×	Slow	Low
SomatoSim	×	✓	×	×	×	×	×	×	×	×	×	Fast	No
stochasticSim	×	✓	×	×	×	✓	×	×	✓	✓	×	Med	Low
ART	✓	×	×	×	×	×	×	×	×	—	×	Fast	No
NEAT	✓	✓	✓	○	×	×	×	×	×	×	×	Med	Yes
ReSeq	✓	×	×	×	×	×	×	×	○	—	✓	Med	Yes
VarSim	✓	✓	✓	✓	×	○	×	×	×	×	×	Med	No
MOV&RSim	✓	✓	✓	✓	✓	✓	×	×	×	○	×	Med	New
GENOMICON-Seq	✓	✓	×	×	×	○	×	×	✓	×	×	Med	New
PSiTE	✓	✓	✓	✓	✓	✓	×	×	×	×	×	Slow	Low
VarForge	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	**Fast**	**Active**

B. Benchmark Configuration Details

Table 4 provides detailed coverage scaling data.

Table 4: Coverage scaling (1 MB reference, 100 mutations, 3 iterations).

Coverage	Read pairs	Wall (s)	Peak RAM (MB)	Pairs/s
1×	6,735	0.25	37	26,940
5×	33,675	1.30	81	25,904
10×	67,350	2.54	137	26,516
30×	202,050	7.18	362	28,140
50×	336,750	11.88	586	28,345
100×	673,335	23.81	1,146	28,279
200×	1,346,670	47.74	2,271	28,209

References

Adalsteinsson, V. A., Ha, G., Freeman, S. S., Choudhury, A. D., Stover, D. G., Parsons, H. A., Gydush, G., Reed, S. C., Rotem, D., Rhoades, J., Loginov, D., Livitz, D., Rosebrock, D., Leshchiner, I., Kim, J., Stewart, C., Rosenberg, M., Francis, J. M., Zhang, C. X., Cohen, O., Oh, C., Ding, H., Polak, P., Lloyd, M., Mahmud, S., Haber, D. A., Belkin, S., Brannigan, B. W., Paik, P. K., Bhatt, D. L., Smasla, B. A., Meyerson, M., Van Allen, E. M., Ellisen, L. W., Iafrate, A. J., Ting, D. T., Brown, M., Deshpande, V., Isselbacher, K. J., Getz, G., Love, J. C., Maheswaran, S., Haber, D. A., and Adalsteinsson, V. A. (2017). Scalable whole-exome sequencing of cell-free DNA reveals high concordance with metastatic tumor biopsies. *Nature Communications*, 8(1):1324. Describes ichorCNA for tumor fraction estimation from low-coverage WGS.

Cristiano, S., Leal, A., Phallen, J., Fiksel, J., Adleff, V., Bruhm, D. C., Jensen, S. O., Medina, J. E., Hruban, C., White, J. R., Palsgrove, D. N., Niknafs, N., Anagnostou, V., Forde, P., Naidoo, J., Marrone, K., Brahmer, J., Woodward, B. D., Husain, H., van Rooijen, K. L., Orntoft, M.-W., Madsen, A. H., van Buuren, M. M., Steeghs, N., Lolkema, M. P., Achterberg, F. B., Mol, L., Velculescu, V. E., and Scharpf, R. B. (2019). Genome-wide cell-free DNA fragmentation in patients with cancer. *Nature*, 570(7761):385–389. Describes DELFI fragmentation-based cancer detection.

Daber, R., Sukhadia, S., and Morrissette, J. J. D. (2021). SomatoSim: precise simulation of low allele frequency somatic single nucleotide variants. *BMC Bioinformatics*, 22:194.

Ewing, A. D., Houlahan, K. E., Hu, Y., Ellrott, K., Caloian, C., Yamaguchi, T. N., Bare, J. C., P'ng, C., Waggott, D., Sabelnykova, V. Y., DREAM Somatic Mutation Calling Challenge Participants, Kellen, M. R., Norman, T. C., Haussler, D., Friend, S. H., Stolovitzky, G., Boutros, P. C., Margolin, A. A., and Stuart, J. M. (2015). Combining tumor genome simulation with crowdsourcing to benchmark somatic single-nucleotide-variant detection. *Nature Methods*, 12(7):623–630. Introduces BAMSurgeon and the ICGC-TCGA DREAM Somatic Mutation Calling Challenge.

Fulcrum Genomics (2016). fgbio: Tools for working with genomic and high throughput sequencing data. https://github.com/fulcrumgenomics/fgbio.

GENOMICON-Seq Authors (2025). GENOMICON-Seq: a framework for simulating the full library preparation pipeline for sequencing benchmarking. *Scientific Reports*. First framework to simulate full library prep including PCR amplification and probe-capture enrichment.

Huang, W., Li, L., Myers, J. R., and Marth, G. T. (2012). ART: a next-generation sequencing read simulator. *Bioinformatics*, 28(4):593–594.

Lo, Y. M. D., Chan, K. C. A., Sun, H., Chen, E. Z., Jiang, P., Lun, F. M. F., Zheng, Y. W., Leung, T. Y., Lau, T. K., Cantor, C. R., and Chiu, R. W. K. (2010). Maternal plasma DNA sequencing reveals the genome-wide genetic and mutational profile of the fetus. *Science Translational Medicine*, 2(61):61ra91.

Luu, M. (2021). noodles: Bioinformatics I/O libraries for Rust. https://github.com/zaeleus/noodles.

MOV&RSim Authors (2025). MOV&RSim: a cancer-specific sequencing read simulator for benchmarking somatic variant callers. *BMC Bioinformatics*, 26. Cancer-specific presets for 21 cancer types; Dockerized.

Mu, J. C., Mohiyuddin, M., Li, J., Bani Asadi, N., Gerstein, M. B., Abyzov, A., Wong, W. H., and Lam, H. Y. K. (2015). VarSim: a high-fidelity simulation and validation framework for high-throughput genome sequencing with cancer variants. *Bioinformatics*, 31(9):1469–1471.

O'Sullivan, B., Doyle, S., Clynes, M., and Barron, N. (2023). Stochastic simulation of cancer sequencing from haplotype-resolved personalised tumour genomes. *NAR Cancer*, 5(3):zcad033. Addresses stochastic VAF sampling and FFPE/oxoG artifacts.

Renaud, G., Hanghoj, K., Korneliussen, T. S., Willerslev, E., and Orlando, L. (2023). NGSNGS: next-generation simulator for next-generation sequencing data. *Bioinformatics*, 39(1).

Schmeing, S. and Robinson, M. D. (2021). ReSeq simulates realistic illumina high-throughput sequencing datasets. *Genome Biology*, 22(1):67.

Schmitt, M. W., Kennedy, S. R., Salk, J. J., Fox, E. J., Hiatt, J. B., and Loeb, L. A. (2012). Detection of ultra-rare mutations by next-generation sequencing. *Proceedings of the National Academy of Sciences*, 109(36):14508–14513. Introduces duplex sequencing methodology.

Smith, T., Heger, A., and Sudbery, I. (2017). UMI-tools: modeling sequencing errors in unique molecular identifiers to improve quantification accuracy. *Genome Research*, 27(3):491–499.

Snyder, M. W., Kircher, M., Hill, A. J., Daza, R. M., and Shendure, J. (2016). Cell-free DNA comprises an in vivo nucleosome footprint that informs its tissues-of-origin. *Cell*, 164(1–2):57–68.

Stephens, Z. D., Hudson, M. E., Mainzer, L. S., Taschuk, M., Weber, M. R., and Iyer, R. K. (2016). Simulating next-generation sequencing datasets from empirical mutation and sequencing

models. *PLOS ONE*, 11(11):e0167047. Describes NEAT (NExt-generation sequencing Analysis Toolkit).

Stone, J. and Matsakis, N. D. (2015). Rayon: A data parallelism library for Rust. https://github.com/rayon-rs/rayon. Version 1.x.

Yang, C., Chu, J., Warren, R. L., and Birol, I. (2019). PSiTE: phylogeny guided simulation of tumour evolution. *Bioinformatics*, 35(23):5065–5067.